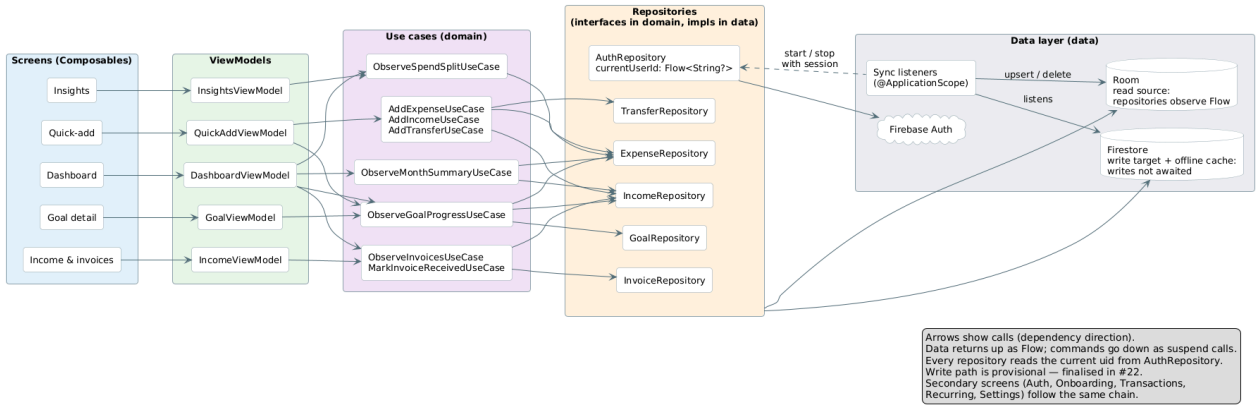
FUNDTRAIL — ARCHITECTURE (MVVM)

Issue: EPIC-003 / T1 (#16) **Use:** Every feature follows the structure and rules in this document. PR review checks against §3 — these are rules, not suggestions.

FundTrail follows strict MVVM with a domain layer.

---

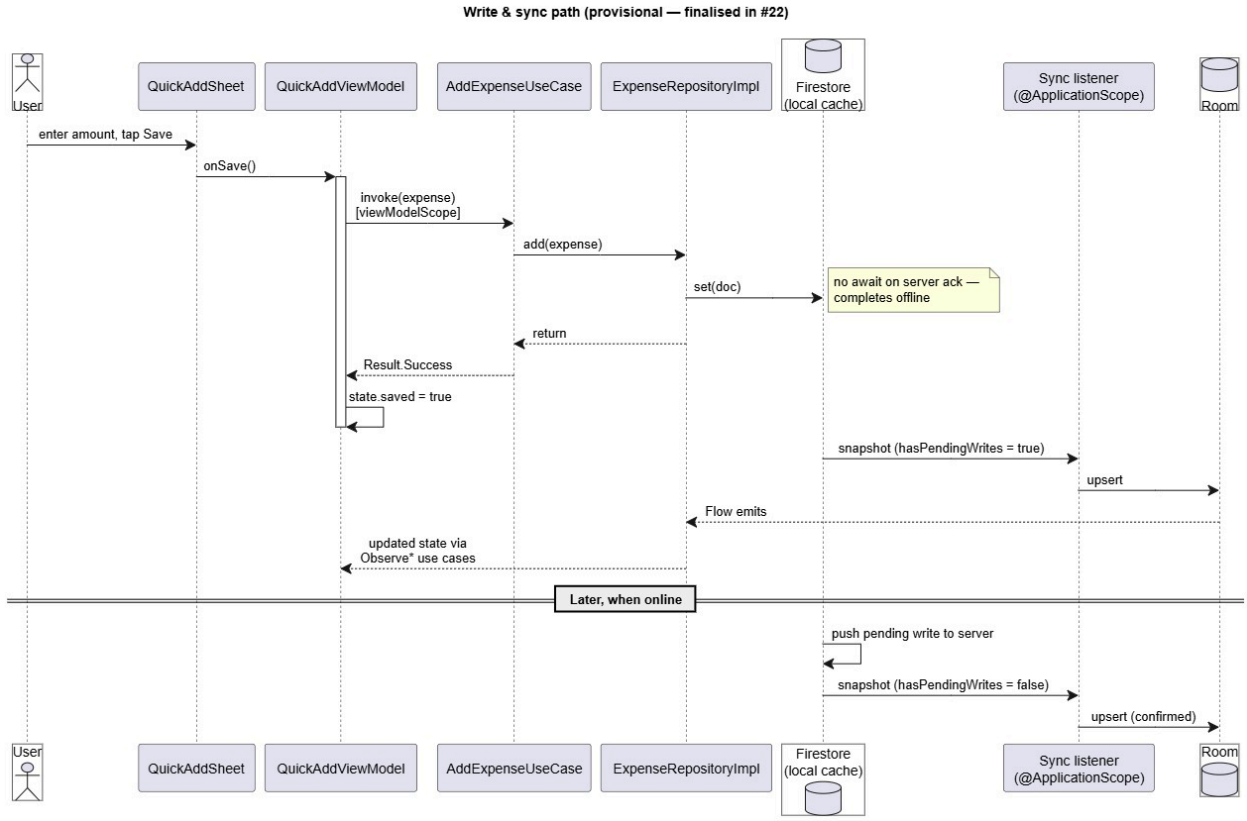
1. ARCHITECTURE DIAGRAM



Data flows up as *Flow*; commands flow down as *suspend* calls. Room is the read source, so screens work offline. Writes go to Firestore, whose snapshot listener syncs changes back into Room.

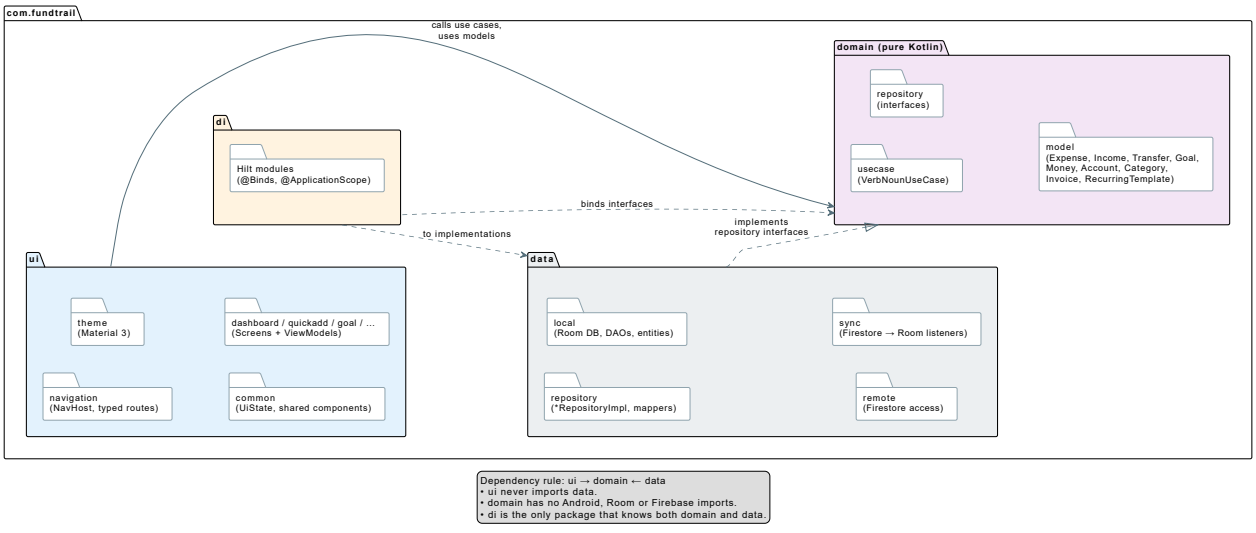
Core daily screens are shown. Secondary screens (Auth, Onboarding, Transactions, Recurring, Settings) follow the same chain.

Write and sync path (rules in §4)



---

2. PACKAGE STRUCTURE



```
com.fundtrail
├── ui                Screens (Composables) and ViewModels, grouped by feature
│   ├── navigation      NavHost, typed routes
│   ├── theme           Material 3 theme
│   ├── common          UiState, shared components (empty, error, loading states)
│   ├── dashboard       DashboardRoute, DashboardScreen, DashboardViewModel
│   ├── quickadd        QuickAddSheet, QuickAddViewModel
│   ├── goal            GoalScreen, GoalViewModel
│   └── …               auth, onboarding, transactions, income, insights, recurring, settings
│
├── domain            Pure Kotlin: no Android, Firebase or Room imports
│   ├── model           Expense, Income, Transfer, Goal, Money, Account, Category,
│   │                   Invoice, RecurringTemplate, Result, …
│   ├── repository      Repository interfaces (incl. AuthRepository)
│   └── usecase         One class per business action, grouped by feature
│
├── data              Implementations of domain interfaces
│   ├── local           Room database, DAOs, entities
│   ├── remote          Firestore access and DTOs
│   ├── repository      *RepositoryImpl classes and mappers
│   └── sync            Firestore listeners that update Room
│
└── di                Hilt modules: bind repository interfaces to implementations;
                      provide the @ApplicationScope CoroutineScope
```

Dependency direction: *ui* → *domain* ← *data*. *ui* never imports *data*. *di* is the only package that knows both sides.

Core domain models

Model	**What it carries**
Money	LKR amount in minor units (*Long*) — the stored truth of what was actually received or spent — plus the optional original currency, amount and rate (display only)
Account	Bank, cash, savings, crypto exchange or other, with an opening balance — every transaction has one
Category	Name + default committed or discretionary kind; each expense keeps its own copy, overridable
Expense / *Income*	*Money*, date, account, status *EXPECTED* or *ACTUAL*; expenses have a category, income has a source (salary, freelance, AdSense, crypto)

Model	What it carries
Transfer	Movement between the user's own accounts — never income or expense
Invoice	Freelance project and milestone, status pending or received; overdue is derived (pending past its due date)
RecurringTemplate	Rent, salary, gym, subscriptions — generates *EXPECTED* entries each period
Goal	Target, deadline and a baseline saved amount; progress is the baseline plus actual net saving since the goal was created (ADR-0006)

Field-level definitions, IDs and invariants are in *Docs/schema.md* (#17).

---

3. RULES

3.1 No logic in Composables

Composables only render state and pass user actions to the ViewModel as lambdas.

- Each screen is split into *XRoute* and *XScreen*. *XRoute* gets the ViewModel with *hiltViewModel()* and collects its state; *XScreen* is stateless and takes only state and lambdas, so it can be previewed with fake state.
- A ViewModel is never created by hand.
- ViewModels only map domain results to UI state and check input format (empty field, unparseable number). Any rule about money, goals, transfers or currency lives in a use case.

3.2 ViewModels expose *StateFlow* only

- Each ViewModel exposes one public *StateFlow* of its screen state. Any *MutableStateFlow* is *private*.
- No *LiveData*, *Channel*, *SharedFlow* or Compose *MutableState* is exposed.
- One-off outcomes such as "saved" or a snackbar message are fields in the state. The UI reports it handled them through a ViewModel callback (e.g. *onMessageShown()*), and the ViewModel clears the field.
- *UiState<T>* is for load-and-display screens. Form screens such as Quick-add use their own state data class.
- ViewModels call use cases only, never repositories.

3.3 *viewModelScope* for async work

- ViewModels launch coroutines only in *viewModelScope*, so work is cancelled when the screen is gone.
- No *GlobalScope* and no *runBlocking*.
- Composables never launch coroutines for data or business work. They may use *LaunchedEffect* / *rememberCoroutineScope* only for Compose UI APIs (e.g. *showSnackbar*,

SheetState.hide(), scroll or animation), and then call back to the ViewModel.
- Long-lived data-layer work (sync listeners) runs in the *@ApplicationScope CoroutineScope* provided by Hilt in *di* — never in a *viewModelScope*.

3.4 Layer rules

- Use cases are named *VerbNounUseCase* and expose a single *operator fun invoke*. Reads return *Flow*: *Observe** passes a repository stream through, *Get** derives or combines one. Writes are *suspend*.
- *ui* never imports *data*. *domain* imports no Android, Room or Firebase classes.
- Repository interfaces return domain models only. Room entities and Firestore DTOs never leave *data*.
- *AuthRepository* exposes *currentUserId: Flow<String?>*. Repositories read the user id from it internally; ViewModels and use cases never pass a user id.
- Repository writes do not await Firestore server acknowledgement. They issue the write and return; the local cache applies it immediately (even offline) and the sync listener updates Room.
- Sync listeners are user-scoped: they start when *currentUserId* becomes non-null and stop when it becomes null. Sign-out is a *SignOutUseCase* that stops sync, then clears Room (full sequence in §4.7).

3.5 Domain rules every use case respects

- The stored LKR amount is the truth. The original currency and rate are for display only and are never used to recompute history.
- Transfers between the user's own accounts are excluded from income and expense totals.
- Only *ACTUAL* income and expenses count toward the month summary and actual saving (Scenario analysis, Lens 1 / A-16). *EXPECTED* entries and pending invoices are shown separately.
- *MarkInvoiceReceivedUseCase* marks the invoice received and creates an *ACTUAL Income* entry in one batch.
- Goal progress is not tied to a single account's balance: it is the goal's baseline saved amount plus actual net saving since the goal was created (ADR-0006; schema §6.3).

---

4. OFFLINE AND SYNC

Issue: EPIC-003 / T4 (#22). Satisfies A-05 (entry works fully offline and syncs later) and the NFR "no blank screen offline".

4.1 Principles

- **Firestore is the source of truth.** Room is a read mirror for the UI and can be rebuilt from Firestore at any time.

- **Only two things write to Room:** the sync listeners (§4.3) and sign-out (§4.7). Repositories and use cases never write Room directly, so there is one path into it and Room can't drift into a second source of truth.
- **Firestore's local cache is the offline queue.** Persistence is on by default on Android; it is never turned off.

4.2 Writes (UI → Firestore)

- Repositories write with *set*, *update* or a *WriteBatch* and do not await the server (§3.4). Firestore applies the write to its local cache at once and queues it on disk. The queue survives app restarts and is sent in order when the device is back online.
- A repository returns *Result.Success* once the write is accepted locally. That is what the user sees as "saved", online or offline.
- **Document IDs are generated on the client** (*collection.document().id*, or the deterministic IDs in *Docs/schema.md* §5), so no write needs a round trip.
- **Writes that must succeed together use one *WriteBatch*** (e.g. invoice receipt, *Docs/schema.md* §6.4). A batch is queued and applied as one unit.
- **No *runTransaction*.** Transactions need the server and fail offline. Use batches, deterministic IDs and *FieldValue.increment* instead.
- **Updates use *update()* or *set(…, SetOptions.merge())* and never resend *createdAt***, which the security rules (#21) keep immutable.
- **Conflicts:** last write wins per field (Firestore's default). Because updates send only the changed fields, two devices editing different fields of one document don't overwrite each other.
- **A write the server rejects** (e.g. by the security rules) is rolled back in Firestore's local cache. The listener then delivers the reverted document, so Room corrects itself. The repository logs the failure; there is no retry UI.

4.3 Sync listeners (Firestore → Room)

- One snapshot listener per subcollection (*accounts*, *categories*, *transactions*, *invoices*, *recurring*, *goals*) plus one on the profile document *users/{uid}*. They live in *data/sync* and run in the *@ApplicationScope* scope, started and stopped with *currentUserId* (§3.4).
- **Listeners are unfiltered** (*Docs/schema.md* §7.1). The first snapshot delivers every document, from the local cache first and then from the server. After that only changes arrive, and Firestore resumes incrementally after a reconnect, so no *updatedAt > lastSync* filter is needed.
- **Each snapshot is applied to Room in one Room transaction:** *ADDED* / *MODIFIED* → upsert; *REMOVED*, or *deletedAt* set → delete the row (*Docs/schema.md* §6.4). Upserts are idempotent, so receiving a document twice is harmless.
- Snapshots are read with *ServerTimestampBehavior.ESTIMATE*, and mappers are null-safe (*Docs/schema.md* §6.5).

- Metadata-only changes are excluded (the default). The app doesn't show per-entry "not synced yet" markers, so it doesn't need them.
- **A listener error** (e.g. *PERMISSION_DENIED*) is logged and recorded in the sync status (§4.5). The UI keeps showing what is in Room. The listener restarts on the next sign-in.

4.4 Room is a cache

- Room holds only the signed-in user's data, so tables carry no user id column.
- **Room schema changes use destructive migration** (*fallbackToDestructiveMigration*). Rebuilding is safe: on the next start the listeners refill Room from Firestore's cache, or from the server. No hand-written Room migrations.
- Room's tables mirror the Firestore collections one to one. Derived values (balances, month totals, goal progress) are computed in queries or use cases, not stored.

4.5 Sync status

The data layer exposes a *SyncStatus* (domain model) through *ObserveSyncStatusUseCase*:

Field	Meaning	Source
initialSyncDone	Every listener has received at least one snapshot from the server (*metadata.isFromCache == false*) for this user.	Stored in a one-row Room table, so it survives restarts and is wiped on sign-out.
isOnline	The device has a validated network connection.	*ConnectivityManager* network callback, behind a domain interface implemented in *data*.
lastError	The most recent listener error, if any.	Sync listeners.

4.6 Screen state rules

Every load-and-display screen maps its data and the sync status to *UiState* with one shared helper in *ui/common*, so all screens behave the same:

Room has data for the screen	***initialSyncDone***	***isOnline***	State shown
yes	—	—	*Success*
no	yes	—	*Empty*: the user really has nothing here yet
no	no	yes	*Loading*: first sync on this device is running
no	no	no	*Error(R.string.offline_no_data)*: "You're offline. Connect once to load your data."

Room has data for the screen	*initialSyncDone*	*isOnline*	State shown
the local read fails	—	—	*Error* from the shared error mapping (#45)

```
// ui/common
fun <T> uiStateOf(hasData: Boolean, sync: SyncStatus, content: () -> T): UiState<T> = when
{
   hasData -> UiState.Success(content())
   sync.initialSyncDone -> UiState.Empty
   sync.isOnline -> UiState.Loading
   else -> UiState.Error(R.string.offline_no_data)
}
```

- **No blank screen offline:** every combination above renders something. A fresh install that has never synced shows the offline message, never an empty "nothing here" screen that would wrongly suggest the data is gone.
- **Offline banner:** one app-wide banner in the main scaffold, driven by *isOnline*. Screens don't each handle connectivity.
- **Content always wins.** When Room has data, the screen shows it, whether online, offline or after a listener error. Sync problems never replace content; a listener error shows as a non-blocking snackbar.
- **Forms never block on connectivity.** Save is never disabled offline, and a save offline looks exactly like a save online (A-05).

4.7 Sign-out

SignOutUseCase runs these steps in order:

1. **Check for unsynced writes.** Call *waitForPendingWrites()* with a short timeout (about 3 s). If it doesn't finish (offline with queued changes), return a result the ViewModel shows as a confirmation: "Some changes haven't synced yet and will be lost if you sign out." Continue only if the user confirms.
2. **Stop the sync listeners**, so none fires with *PERMISSION_DENIED* once auth is gone.
3. ***FirebaseAuth.signOut()*.**
4. **Clear Firestore's local cache:** *terminate()*, then *clearPersistence()*. The disk cache holds the previous user's documents and any unsent writes, so clearing Room alone would leave them on the device.
5. **Clear Room:** *clearAllTables()*, which also wipes the sync status row.
6. Navigate to the auth graph with the back stack cleared (#51).

After *terminate()* the old *FirebaseFirestore* instance can't be used, and *FirebaseFirestore.getInstance()* returns a new one. The Hilt module therefore provides

FirebaseFirestore **unscoped**, and data classes inject *Provider<FirebaseFirestore>* and call *get()* for each operation instead of keeping an instance.

4.8 Out of scope

Per-entry "not synced yet" markers, a retry UI for rejected writes, and showing the number of queued changes. None is needed for A-05.

EXAMPLE

```
// ui/common
sealed interface UiState<out T> {
    data object Loading : UiState<Nothing>
    data object Empty : UiState<Nothing>
    data class Error(@StringRes val message: Int) : UiState<Nothing>
    data class Success<T>(val data: T) : UiState<T>
}

// ui/dashboard
@HiltViewModel
class DashboardViewModel @Inject constructor(
    observeMonthSummary: ObserveMonthSummaryUseCase,
    observeGoalProgress: ObserveGoalProgressUseCase,
    observeSpendSplit: ObserveSpendSplitUseCase,
    observeSyncStatus: ObserveSyncStatusUseCase,
) : ViewModel() {

    val uiState: StateFlow<UiState<DashboardContent>> =
        combine(observeMonthSummary(), observeGoalProgress(), observeSpendSplit(),
observeSyncStatus()) { summary, goal, split, sync ->
            val hasData = !(summary.hasNoEntries && goal == null)
            uiStateOf(hasData, sync) { DashboardContent(summary, goal, split) } // §4.6
        }
            .catch { emit(UiState.Error(R.string.error_dashboard)) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), UiState.Loading)
}

@Composable
fun DashboardRoute(viewModel: DashboardViewModel = hiltViewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    DashboardScreen(state = state)
}
```

The example needs *lifecycle-runtime-compose* (*collectAsStateWithLifecycle*) and *hilt-navigation-compose* (*hiltViewModel*). Retry after an error is out of scope here.